



Интро

- ТЕХНИКА

Проверка связи

Если у вас нет звука

убедитесь, что на устройстве и колонках включён звук
обновите страницу или переподключитесь к вебинару
откройте вебинар в другом браузере
перезагрузите устройство и войдите заново

Пожалуйста, отметьтесь в чате



если меня видно и слышно



если есть проблемы со звуком или видео

- 01 / Десктоп

С компьютера

Используйте **Google Chrome** или **Microsoft Edge**.

При проблемах - обновите страницу **F5**

- 02 / Мобильный

С телефона или планшета

Перейдите с мобильного интернета на **Wi-Fi**.

При проблемах - перезапустите приложение Webinar.

- 03 / Общий совет

Если ничего не помогло

Отключитесь и подключитесь заново.

Пишите в чат - организаторы помогут.

- ЗНАКОМСТВО

Перед началом

- КАКИЕ НОВОСТИ?

Что интересного произошло за неделю?

Напишите в чат - событие, новость или вывод из работы, учёбы или личного опыта.



- ПРЕПОДАВАТЕЛЬ КУРСА

Павлин Николай

Преподаватель кафедры бизнес-информатики МТУСИ
с 2021 года

СТО Nova · стартап в сфере мониторинга здоровья · резидент Сколково

Разрабатываю **LLM-приложения** и RAG-платформу **RAGMaster** (грант ФСИ)

Автор RAG-системы для охраны труда - **chat.kiout.ru**, 1000+ пользователей

Соавтор LLM, попавшей в **топ-20** русскоязычных моделей по MERA (02.2025)

7+ лет в коммерческой разработке: Python, FastAPI, LLM-стек, DevOps

Многократный победитель федеральных хакатонов



Telegram



YouTube



LinkedIn

- ДОГОВОРЁННОСТИ

Правила участия

- 1** Продолжительность вебинара - **80 минут**
- 2** Задавайте **вопросы в чате** - ответу в процессе или в конце
- 3** **Запись** будет доступна на платформе на следующий день
- 4** Обсуждение можно **продолжить в чате** после вебинара
- 5** **Воздерживаемся от обсуждения политических тем.** Здесь мы про разработку, ИИ и продукты - давайте сохраним фокус.

Начинаем!





FastAPI-сервис для LLM

Модуль 3 Блок 4

Павлин Н.К.

План занятия

Блок	Что делаем
Теория	Почему FastAPI для LLM: async, pydantic, OpenAPI
Теория	Структура проекта, pydantic-settings v2, схемы
Теория	DI, lifespan, middleware, exception handlers

Что вы уже знаете — опираемся на это

Что добавляем сегодня:

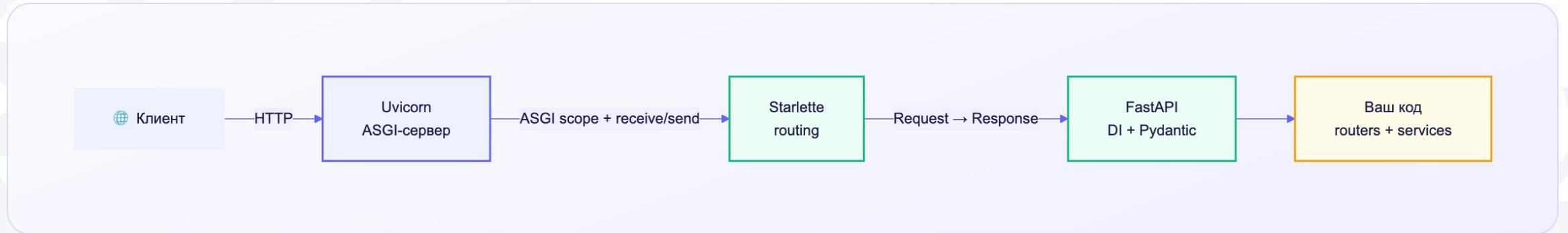
- HTTP-фасад над async-клиентом (FastAPI)
- Production-структура с разделением ответственности
- DI вместо глобалов
- **lifespan** для управления жизненным циклом
- Автоматический Swagger и тесты с подменой зависимостей

Откуда	Что берём в сегодняшнем сервисе
M2B1	Базовый API-клиент: ключ, выбор модели, базовые вызовы
M2B3	Retry с <code>tenacity</code> , fallback-цепочка провайдеров
M2B4	Кеш ответов в Redis
M3B2	Слои Gateway / Service / LLM / Data, Circuit Breaker, Bulkhead
M3B3	<code>AsyncOpenAI</code> , <code>asyncio.gather</code> , <code>Semaphore</code> , SSE через async generator

Словарь занятия

Термин	English	Что это простыми словами
ASGI	ASGI	Аsync-интерфейс между веб-сервером и Python-приложением. Преемник WSGI
Uvicorn	Uvicorn	ASGI-сервер на базе uvloop. Запускает наш FastAPI-процесс
Gunicorn	Gunicorn	Process manager. Держит N воркеров Uvicorn, переподнимает упавшие
APIRouter	APIRouter	Группа endpoints одного домена: <code>chat</code> , <code>models</code> , <code>health</code>
Depends	Depends	Маркер, что аргумент endpoint — это зависимость, которую создаст FastAPI
Annotated[T, Depends(...)]	Annotated	Современный способ описать тип + DI одной конструкцией (PEP 593)
lifespan	lifespan	Аsync-контекстный менеджер на старт/стоп приложения (FastAPI 0.93+)
BaseModel	Pydantic BaseModel	Класс для описания схемы данных с валидацией
BaseSettings	pydantic-settings	Читает конфиг из env / .env с валидацией типов
Discriminated Union	Discriminated Union	Union типов с дискриминатором — Pydantic выбирает класс по полю <code>type</code>
StreamingResponse	StreamingResponse	Ответ, собираемый из аsync-генератора. Основа SSE
SSE	Server-Sent Events	HTTP-поток <code>text/event-stream</code> с событиями <code>data: ...\n\n</code>
httpx	httpx	Аsync HTTP-клиент. Используем в тестах как <code>AsyncClient</code>
pytest-asyncio	pytest-asyncio	Плагин pytest для аsync-тестов. Режим <code>asyncio_mode = "auto"</code>
tenacity	tenacity	Библиотека retry с декораторами: экспонента, jitter, условия
dependency_overrides	dependency_overrides	Механизм FastAPI: подменить Depends-зависимость в тестах

ASGI, Uvicorn, FastAPI: как это устроено



ASGI, Uvicorn, FastAPI: как это устроено

Слои:

- **ASGI** — интерфейс async-сервера и приложения. PEP 3333 для async-мира
- **Uvicorn** — реализация ASGI-сервера на [uvloop](#) + [httptools](#)
- **Starlette** — micro-фреймворк с роутингом, middleware, WebSocket
- **FastAPI** — надстройка над Starlette: DI, валидация через Pydantic, OpenAPI

Важно: FastAPI = Starlette + магия. Всё, что вы знаете про Starlette (Request, Response, middleware), работает в FastAPI напрямую.

Почему именно FastAPI для LLM-сервиса

Четыре причины:

- **Async-native.** `async def` — первоклассный гражданин. Для LLM, где 99% времени — ожидание ответа, это критично
- **Pydantic-валидация из коробки.** Пришёл JSON → он уже провалидирован, типизирован, сериализован
- **Автоматический OpenAPI/Swagger.** Фронт, партнёры, QA — все видят контракт без усилий
- **Зрелая экосистема.** Uvicorn, Starlette, sse-starlette, pydantic-settings — всё работает друг с другом из коробки

Альтернативы и когда имеет смысл:

- **Flask + Flask-Async** — годится для старых кодовых баз, но `async` прикручен сбоку
- **Django REST Framework** — если у вас уже есть Django-админка и ORM. Иначе тяжело
- **LiteStar** — растущий конкурент FastAPI, но экосистема меньше
- **aiohhttp / Starlette напрямую** — когда нужна максимальная кастомизация и вам не нужен «магический» DI

Для курса и для дипломного проекта — **FastAPI**.

Конфигурация через окружение

Почему не `config.yaml` / `config.py` с хардкодом:

- **Секреты** — ключ OpenAI не должен лежать в git
- **Окружения** — dev / staging / prod имеют разные ключи, базы, URLs
- **Docker / Kubernetes** — стандарт: env-переменные + секреты через volumes
- **12-factor** — индустриальный стандарт для cloud-native приложений

Иерархия источников (от приоритета):

1. Переменные окружения процесса (`export OPENAI_API_KEY=...`)
2. `.env` файл в корне проекта (для dev)
3. Значения по умолчанию в коде (`max_tokens: int = 1024`)
4. Отсутствие обязательного поля → `ValidationError` при старте

`pydantic-settings` поддерживает всю эту иерархию из коробки.

Опрос: как вы сейчас храните конфиг

Варианты:

- Переменные окружения через `os.getenv("OPENAI_API_KEY")`
- `.env` + `python-dotenv`
- Файл `settings.py` или `constants.py` с хардкодом
- `pydantic-settings` / `BaseSettings`
- Vault / AWS Parameter Store / Secrets Manager
- Что-то другое — опишите в чате

Структура production-проекта

```
llm-service/  
├── app/  
│   ├── __init__.py  
│   ├── main.py           # FastAPI app, lifespan, include_router  
│   └── core/  
│       ├── config.py     # Settings (pydantic-settings)  
│       ├── logging.py    # structlog (БЗ.6)  
│       └── exceptions.py  # Кастомные исключения домена  
├── deps/  
│   └── providers.py       # get_llm_client, get_cache, get_settings  
├── routers/  
│   ├── chat.py           # /chat, /chat/stream, /chat/batch  
│   ├── models.py         # /models  
│   └── health.py         # /health, /ready  
├── services/  
│   ├── llm.py            # Вызовы LLM, fallback, retry  
│   └── cache.py          # Redis через aiocache  
├── schemas/  
│   ├── chat.py           # ChatRequest, ChatResponse, Message  
│   └── models.py         # ModelInfo  
├── tests/  
│   ├── conftest.py       # Фикстуры: AsyncClient + моки  
│   ├── test_chat.py  
│   └── test_models.py  
├── .env                  # HE в git  
├── .env.example          # Шаблон — в git  
├── pyproject.toml        # uv / poetry / pdm  
└── README.md
```


Почему именно такая структура

Три причины:

- **Тестируемость.** Router тестируем с замканным service. Service тестируем с замканным LLM-клиентом. Ничего не нужно поднимать целиком
- **Расширяемость.** Новый endpoint — новый файл в `routers/`. Не трогаем существующие
- **Командная работа.** Параллельные ветки в разных папках почти не конфликтуют

Чего мы избегаем:

- **Монофайл `main.py` на 500 строк** — быстро превращается в ад при отладке
- **Утечка HTTP в сервисы** — если `LLMService` принимает `Request` — это плохо, сервис становится неюзабельным из CLI, из бота, из cron-задачи
- **Утечка бизнес-логики в схемы** — Pydantic-модели не должны ходить в Redis или вызывать OpenAI

Переходим к коду: конфиг и схемы

Закончили с обзором. Дальше — конкретный код, на котором соберём сервис.

Что было: контекст, словарь, почему FastAPI, структура папок.

Что добавляем: `pydantic-settings` для конфига, `Pydantic` схемы запроса/ответа, discriminated union для мульти-провайдера.

pydantic-settings v2: блок LLM

```
example.python

1  # app/core/config.py
2  from pydantic import SecretStr
3  from pydantic_settings import BaseSettings, SettingsConfigDict
4
5
6  class LLMSettings(BaseSettings):
7      model_config = SettingsConfigDict(env_prefix="LLM_")
8
9      openai_api_key: SecretStr
10     default_model: str = "gpt-5-mini"
11     request_timeout: float = 30.0
12     max_retries: int = 3
```

pydantic-settings v2: корневой Settings

example.python

```
1 # app/core/config.py (продолжение)
2 from functools import lru_cache
3 from pydantic import Field
4 from pydantic_settings import BaseSettings, SettingsConfigDict
5
6
7 class Settings(BaseSettings):
8     model_config = SettingsConfigDict(
9         env_file=".env",
10         env_file_encoding="utf-8",
11         env_nested_delimiter="__",
12         extra="ignore",
13     )
14
15     app_name: str = "llm-service"
16     debug: bool = False
17     cors_origins: list[str] = Field(default_factory=lambda: ["*"])
18     redis_url: str = "redis://localhost:6379/0"
19     cache_ttl_seconds: int = 3600
20     llm: LLMSettings = Field(default_factory=LLMSettings)
```

example.python

```
1 @lru_cache
2 def get_settings() -> Settings:
3     return Settings()
```

Задача: ChatRequest для вашего проекта

Напишите в чат или в код: какие поля должен принимать ваш `/chat`-endpoint?

Обязательный минимум:

- `messages: list[Message]`

Вопросы, на которые нужно ответить:

- Поддерживаете ли вы несколько провайдеров? Как выбирает endpoint?
- Есть ли `user_id` для логов и rate-limit?
- Есть ли `session_id` для мультиходового диалога?
- Нужен ли `stream: bool` или отдельный endpoint?
- `response_format`: free text / JSON / structured output?

Pydantic-схемы для LLM endpoints

example.python

```
1 # app/schemas/chat.py
2 from typing import Annotated, Literal
3 from pydantic import BaseModel, Field, model_validator
4
5 Role = Literal["system", "user", "assistant", "tool"]
6
7
8 class Message(BaseModel):
9     role: Role
10     content: Annotated[str, Field(min_length=1, max_length=100_000)]
11
12
13 class ChatRequest(BaseModel):
14     messages: Annotated[list[Message], Field(min_length=1, max_length=50)]
15     model: str = "gpt-5-mini"
16     temperature: Annotated[float, Field(ge=0.0, le=2.0)] = 0.7
17     max_tokens: Annotated[int, Field(ge=1, le=16_000)] = 1024
18     stream: bool = False
19     user_id: str | None = None
```

example.python

```
1 @model_validator(mode="after")
2 def _first_message_not_assistant(self) -> "ChatRequest":
3     if self.messages[0].role == "assistant":
4         raise ValueError("Первое сообщение не может быть от assistant")
5     return self
```

ChatResponse: что возвращаем клиенту

example.python

```
1 # app/schemas/chat.py (продолжение)
2 from pydantic import BaseModel
3
4
5 class Usage(BaseModel):
6     prompt_tokens: int
7     completion_tokens: int
8     total_tokens: int
9     estimated_cost_usd: float = 0.0
10
11 @classmethod
12 def from_openai(cls, u) -> "Usage":
13     return cls(
14         prompt_tokens=u.prompt_tokens,
15         completion_tokens=u.completion_tokens,
16         total_tokens=u.total_tokens,
17     )
```

example.python

```
1 class ChatResponse(BaseModel):
2     content: str
3     model: str
4     usage: Usage
5     finish_reason: str | None = None
6     cached: bool = False
7     request_id: str | None = None
8
9 @classmethod
10 def from_openai(cls, raw) -> "ChatResponse":
11     choice = raw.choices[0]
12     return cls(
13         content=choice.message.content,
14         model=raw.model,
15         usage=Usage.from_openai(raw.usage),
16         finish_reason=choice.finish_reason,
17     )
```


Discriminated Union: мульти-провайдер в одном endpoint

example.python

```
1 # app/schemas/chat.py (продолжение)
2 from typing import Literal, Union, Annotated
3 from pydantic import BaseModel, Field
4
5
6 class OpenAIParams(BaseModel):
7     provider: Literal["openai"]
8     model: str = "gpt-5-mini"
9     seed: int | None = None
10
11
12 class OllamaParams(BaseModel):
13     provider: Literal["ollama"]
14     model: str = "llama3.1:8b"
15     base_url: str = "http://localhost:11434/v1"
16
17
18 ProviderParams = Annotated[
19     Union[OpenAIParams, OllamaParams],
20     Field(discriminator="provider"),
21 ]
```

example.python

```
1 class ChatRequestMulti(BaseModel):
2     messages: list[Message]
3     params: ProviderParams
4     temperature: float = 0.7
5     max_tokens: int = 1024
```


Discriminated Union: мульти-провайдер в одном endpoint

Зачем: разные провайдеры принимают разные параметры (у OpenAI — `seed`, у локального Ollama — `base_url` на свой сервер). Discriminated union даёт типизированный выбор на уровне схемы.

APIRouter: разделение по доменам

example.python

```
1 # app/routers/chat.py
2 from fastapi import APIRouter
3
4 router = APIRouter(prefix="/chat", tags=["chat"])
5
6
7 @router.post("")
8 async def chat_completions(...): ...
9
10
11 @router.post("/stream")
12 async def chat_stream(...): ...
13
14
15 @router.post("/batch")
16 async def chat_batch(...): ...
```

example.python

```
1 # app/routers/models.py
2 from fastapi import APIRouter
3
4 router = APIRouter(prefix="/models", tags=["models"])
5
6
7 @router.get("")
8 async def list_models(): ...
```

APIRouter: разделение по доменам

```
example.python

1 # app/routers/health.py
2 from fastapi import APIRouter
3
4 router = APIRouter(tags=["health"])
5
6
7 @router.get("/health")
8 async def health(): return {"status": "ok"}
9
10
11 @router.get("/ready")
12 async def ready(): ...
```

lifespan vs устаревший on_event

Было (deprecated):

```
example.python

1 @app.on_event("startup")
2 async def startup():
3     app.state.llm = AsyncOpenAI(api_key=...)
4
5 @app.on_event("shutdown")
6 async def shutdown():
7     await app.state.llm
```

lifespan vs устаревший on_event

Стало (FastAPI 0.93+):

```
example.python

1 from contextlib import asynccontextmanager
2
3 @asynccontextmanager
4 async def lifespan(app: FastAPI):
5     app.state.llm = AsyncOpenAI(api_key=...)
6     yield
7     await app.state.llm.close()
8
9 app = FastAPI(lifespan=lifespan)
```

lifespan vs устаревший on_event

Преимущества lifespan:

- Один `async with` вместо двух callback'ов — ресурсы как в `contextlib`
- `try / finally` для cleanup работает корректно даже при падении старта
- Async-safe без отдельных флагов `is_started`

lifespan: инициализация клиента и Redis — код

Один `async-context-manager`. Всё до `yield` — `startup`, после — `shutdown`.

example.python

```
1 # app/main.py
2 from contextlib import asynccontextmanager
3 from fastapi import FastAPI
4 from openai import AsyncOpenAI
5 from redis.asyncio import Redis
6
7 from app.core.config import get_settings
8 from app.routers import chat, models, health
9
10
11 @asynccontextmanager
12 async def lifespan(app: FastAPI):
13     settings = get_settings()
14
15     app.state.llm = AsyncOpenAI(
16         api_key=settings.llm.openai_api_key.get_secret_value(),
17         timeout=settings.llm.request_timeout,
18         max_retries=settings.llm.max_retries,
19     )
20     app.state.redis = Redis.from_url(settings.redis_url, decode_responses=True)
21
22     yield
```

example.python

```
1     await app.state.llm.close()
2     await app.state.redis.close()
3
4
5 app = FastAPI(title="LLM Service", version="1.0.0", lifespan=lifespan)
6 app.include_router(chat.router)
7 app.include_router(models.router)
8 app.include_router(health.router)
```

lifespan: разбор и подводные камни

По шагам:

- Всё до `yield` — startup. FastAPI ждёт завершения перед тем, как принять первый запрос
- `yield` — приложение живёт и обрабатывает HTTP
- Всё после `yield` — graceful shutdown. Uvicorn сначала отказывает в новых соединениях, ждёт текущие, потом вызывает shutdown
- `app.state` — стандартное хранилище на весь lifetime приложения, привязано к FastAPI-инстансу, не глобальная переменная

Грбли:

- Создавать `AsyncOpenAI()` на уровне модуля (`import time`) — event loop ещё не создан, `httpx` падает в некоторых версиях. `lifespan` — правильное место
- Старый `@app.on_event("startup")` — deprecated, будет удалён
- В тестах через `AsyncClient(transport=ASGITransport(app))` `lifespan` запускается. Если не нужен — `lifespan="off"` в `TestClient`

Dependency Injection: Annotated + Depends

example.python

```
1 # app/deps/providers.py
2 from typing import Annotated
3 from fastapi import Depends, Request
4 from openai import AsyncOpenAI
5 from redis.asyncio import Redis
6
7 from app.core.config import Settings, get_settings
8 from app.services.llm import LLMService
9
10
11 SettingsDep = Annotated[Settings, Depends(get_settings)]
12
13
14 def get_llm(request: Request) -> AsyncOpenAI:
15     return request.app.state.llm
16
17
18 def get_cache(request: Request) -> Redis:
19     return request.app.state.redis
20
21
22 LLMDep = Annotated[AsyncOpenAI, Depends(get_llm)]
23 CacheDep = Annotated[Redis, Depends(get_cache)]
```

example.python

```
1
2 def get_llm_service(llm: LLMDep, cache: CacheDep) -> LLMService:
3     return LLMService(llm=llm, cache=cache)
4
5
6 LLMServiceDep = Annotated[LLMService, Depends(get_llm_service)]
```

Dependency Injection: Annotated + Depends

Использование в endpoint:

example.python

```
1 @router.post("")
2 async def chat_completions(req: ChatRequest, service: LLMServiceDep) -> ChatResponse:
3     return await service.complete(req)
```

Sub-dependencies и yield-зависимости

Зависимости могут зависеть от зависимостей:

```
example.python

1 def get_user_id(request: Request) -> str:
2     return request.headers.get("X-User-Id", "anonymous")
3
4
5 def get_rate_limiter(
6     user_id: Annotated[str, Depends(get_user_id)],
7     cache: CacheDep,
8 ) -> RateLimiter:
9     return RateLimiter(user_id=user_id, cache=cache, limit=60, window=60)
10
11
12 RateLimiterDep = Annotated[RateLimiter, Depends(get_rate_limiter)]
```

Sub-dependencies и yield-зависимости

Зависимости с очисткой через yield:

```
example.python

1  async def get_db_session() -> AsyncIterator[AsyncSession]:
2      async with SessionMaker() as session:
3          try:
4              yield session
5              await session.commit()
6          except Exception:
7              await session.rollback()
8              raise
```

Endpoints: разбор кода

Конфиг, схемы и DI — собраны. Теперь главное содержание занятия: пять endpoints.

Что было: Settings, ChatRequest/Response, APIRouter, lifespan, DI через Annotated.

Что добавляем: `/chat`, `/chat/stream` (SSE), `/chat/batch`, `/models`, `/health` — с сервисным слоем.

Endpoint /chat: синхронный запрос

```
example.python

1  # app/routers/chat.py
2  from fastapi import APIRouter
3
4  from app.schemas.chat import ChatRequest, ChatResponse
5  from app.deps.providers import LLMServiceDep
6
7  router = APIRouter(prefix="/chat", tags=["chat"])
8
9
10 @router.post(
11     "",
12     response_model=ChatResponse,
13     summary="Синхронный чат",
14     description="Отправляет сообщения в LLM и возвращает полный ответ.",
15 )
16 async def chat_completions(req: ChatRequest, service: LLMServiceDep) -> ChatResponse:
17     return await service.complete(req)
```

Endpoint /chat: синхронный запрос

Что делает FastAPI автоматически:

- Валидирует JSON → `ChatRequest`. Ошибка → `422 Unprocessable Entity`
- Инжектит `service` через DI-граф (`LLMService` → `LLM` + `Cache`)
- Вызывает endpoint (уже `async`, блокировать loop нельзя)
- Сериализует результат в JSON по `response_model`

Сервис: вызов LLM + кеш + retry — код

Сервис собирает кеш (M2Б4), retry (M2Б3) и async-клиент (M3Б3) в одну точку.

```
example.python
1 # app/services/llm.py
2 import hashlib, json
3 from openai import AsyncOpenAI
4 from redis.asyncio import Redis
5 from tenacity import retry, stop_after_attempt, wait_exponential
6
7 from app.schemas.chat import ChatRequest, ChatResponse
8
9
10 class LLMService:
11     def __init__(self, llm: AsyncOpenAI, cache: Redis, ttl: int = 3600):
12         self.llm = llm
13         self.cache = cache
14         self.ttl = ttl
```

```
example.python
1 def _key(self, req: ChatRequest) -> str:
2     payload = req.model_dump(exclude={"user_id", "stream"})
3     blob = json.dumps(payload, sort_keys=True, ensure_ascii=False)
4     return "chat:" + hashlib.sha256(blob.encode()).hexdigest()
5
6 @retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=10))
7 async def _call(self, req: ChatRequest) -> ChatResponse:
8     raw = await self.llm.chat.completions.create(
9         model=req.model,
10         messages=[m.model_dump() for m in req.messages],
11         temperature=req.temperature,
12         max_tokens=req.max_tokens,
13     )
14     return ChatResponse.from_openai(raw)
```


Сервис: вызов LLM + кеш + retry — код

```
example.python

1  async def complete(self, req: ChatRequest) -> ChatResponse:
2      if req.temperature > 0:
3          resp = await self._call(req)
4          resp.cached = False
5          return resp
6
7      key = self._key(req)
8      if blob := await self.cache.get(key):
9          resp = ChatResponse.model_validate_json(blob)
10         resp.cached = True
11         return resp
12
13     resp = await self._call(req)
14     resp.cached = False
15     await self.cache.setex(key, self.ttl, resp.model_dump_json())
16     return resp
```

Сервис: разбор по строкам

Ключевые места кода:

- `_key()` — хеш от нормализованного запроса (без `user_id` и `stream`), чтобы разные пользователи попадали в один кеш при одинаковом промпте
- `@retry(...)` — берём `tenacity`, свой `retry` не пишем
- `setex(key, ttl, ...)` — атомарная установка с TTL. Без TTL кеш разбухнет
- `model_validate_json / model_dump_json` — Pydantic v2 сериализация. Быстрее, чем `json.loads + model_validate`
- `complete()` ветка `if req.temperature > 0` — кешируем только детерминированные ответы

На что обратить внимание:

- Кеш по умолчанию хешит весь `request`. При `temperature=0.9` это ломает вариативность — поэтому ветка
- `exclude={"user_id", "stream"}` — для кеша важен контент промпта, не метаданные
- `Retry` внутри `_call`, до записи в кеш — кешируем только успешные ответы

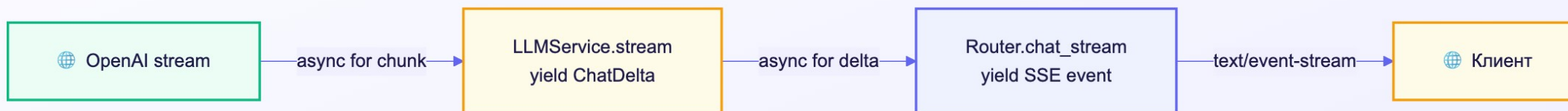
Напоминание: async-генераторы из Б3.3

Из Б3.3 вы уже писали:

```
example.python  
  
1  async for chunk in stream:  
2      print(chunk.choices[0].delta.content, end="")
```

Напоминание: async-генераторы из БЗ.3

Сегодня мы это же отдаём как HTTP-поток. Сервис — async-generator. Endpoint — обёртка для SSE.



Напоминание: async-генераторы из БЗ.3

Три уровня асинхронности:

- **OpenAI SDK** — отдаёт `AsyncStream[ChatCompletionChunk]`
- **Наш сервис** — `AsyncIterator[ChatDelta]` (domain-объект)
- **Endpoint** — `AsyncIterator[dict]` (SSE-событие)

Endpoint /chat/stream: SSE через StreamingResponse

```
example.python

1  # app/routers/chat.py (продолжение)
2  from fastapi.responses import StreamingResponse
3
4  from app.deps.providers import LLMServiceDep
5
6
7  @router.post("/stream", summary="Streaming чат через SSE")
8  async def chat_stream(req: ChatRequest, service: LLMServiceDep):
9      async def event_source():
10         async for delta in service.stream(req):
11             yield f"data: {delta.model_dump_json()}\n\n"
12             yield "data: [DONE]\n\n"
13
14     return StreamingResponse(
15         event_source(),
16         media_type="text/event-stream",
17         headers={"X-Accel-Buffering": "no", "Cache-Control": "no-cache"},
18     )
```

Endpoint /chat/stream: SSE через StreamingResponse

Что важно в этом коде:

- `media_type="text/event-stream"` — корректный Content-Type для SSE
- Формат `data: <текст>\n\n` собираем руками — это и есть протокол SSE
- `[DONE]` — терминатор по конвенции OpenAI; клиент видит, что поток закончился
- `X-Accel-Buffering: no` — выключаем буферизацию nginx, иначе SSE «склеит» все события в один блок

LLMService.stream: генератор под SSE

```
example.python

1  # app/services/llm.py (продолжение)
2  from collections.abc import AsyncIterator
3  from app.schemas.chat import ChatDelta
4
5
6  class LLMService:
7      # ... init, complete, _call как выше
8
9      async def stream(self, req: ChatRequest) -> AsyncIterator[ChatDelta]:
10         stream = await self.openai.chat.completions.create(
11             model=req.model,
12             messages=[m.model_dump() for m in req.messages],
13             temperature=req.temperature,
14             max_tokens=req.max_tokens,
15             stream=True,
16             stream_options={"include_usage": True},
17         )
18         async for chunk in stream:
19             if chunk.choices and (delta := chunk.choices[0].delta).content:
20                 yield ChatDelta(content=delta.content)
21             if chunk.usage:
22                 yield ChatDelta(usage=Usage.from_openai(chunk.usage))
```


Endpoint /chat/batch: parallel через gather

```
example.python

1  # app/routers/chat.py (продолжение)
2  import asyncio
3  from fastapi import HTTPException
4
5  from app.deps.providers import LLMServiceDep
6
7  BATCH_SEM = asyncio.Semaphore(5)
8
9
10 @router.post("/batch", summary="Batch чат: несколько запросов за раз")
11 async def chat_batch(
12     reqs: list[ChatRequest],
13     service: LLMServiceDep,
14 ) -> list[ChatResponse | dict]:
15     if len(reqs) > 20:
16         raise HTTPException(status_code=413, detail="Максимум 20 запросов в batch")
```

Endpoint /chat/batch: parallel через gather

example.python

```
1     async def _one(r: ChatRequest):
2         async with BATCH_SEM:
3             try:
4                 return await service.complete(r)
5             except Exception as e:
6                 return {"error": type(e).__name__, "detail": str(e)}
7
8     return await asyncio.gather(*(_one(r) for r in reqs))
```

Endpoint /models: discovery

example.python

```
1  # app/routers/models.py
2  from fastapi import APIRouter
3
4  from app.schemas.models import ModelInfo
5  from app.deps.providers import SettingsDep
6
7  router = APIRouter(prefix="/models", tags=["models"])
8
9  CATALOG = {
10     "gpt-5-mini": ModelInfo(id="gpt-5-mini", provider="openai", input_per_lm=0.40, output_per_lm=1.60),
11     "gpt-5.2":    ModelInfo(id="gpt-5.2",    provider="openai", input_per_lm=2.00, output_per_lm=8.00),
12     "gpt-5":      ModelInfo(id="gpt-5",      provider="openai", input_per_lm=5.00, output_per_lm=20.00),
13 }
14
15
16 @router.get("", response_model=list[ModelInfo])
17 async def list_models(settings: SettingsDep) -> list[ModelInfo]:
18     return list(CATALOG.values())
```

Сквозные слои: middleware, ошибки, кеш

Endpoints написаны. Теперь — то, что общее для всех endpoints.

Что было: пять endpoints с собственной логикой.

Что добавляем: middleware (request_id, latency, cost, CORS), exception handlers (домен → HTTP), кеш Redis, Swagger.

Middleware observability: код

Один middleware — три ответственности: request_id, latency, cost.

```
example.python

1 # app/main.py (продолжение)
2 import time, uuid, logging
3 from fastapi import Request
4
5 logger = logging.getLogger("llm-service")
6
7
8 @app.middleware("http")
9 async def observability_middleware(request: Request, call_next):
10     request.state.request_id = request.headers.get("X-Request-ID", uuid.uuid4().hex)
11     request.state.llm_cost = 0.0
12     request.state.llm_tokens = 0
13
14     t0 = time.perf_counter()
15     try:
16         response = await call_next(request)
17     except Exception:
18         logger.exception("unhandled", extra={"request_id": request.state.request_id})
19         raise
```

Middleware observability: код

```
example.python
1 duration_ms = (time.perf_counter() - t0) * 1000
2 response.headers["X-Request-ID"] = request.state.request_id
3 response.headers["X-LLM-Cost-USD"] = f"{request.state.llm_cost:.6f}"
4 logger.info(
5     "request",
6     extra={
7         "request_id": request.state.request_id,
8         "method": request.method,
9         "path": request.url.path,
10        "status": response.status_code,
11        "duration_ms": round(duration_ms, 2),
12        "llm_cost": request.state.llm_cost,
13        "llm_tokens": request.state.llm_tokens,
14    },
15 )
16 return response
```

Middleware observability: разбор

Что важно по строкам:

- `request.state` — словарь, живёт весь запрос. Middleware кладёт `request_id`, сервисы пишут `llm_cost/llm_tokens`
- `X-Request-ID` — сквозной ID для логов и трейсов. Принимаем от клиента, иначе генерируем. Отдаём обратно в заголовке
- `X-LLM-Cost-USD` в response — фронт видит стоимость запроса сразу, без отдельного запроса
- `time.perf_counter()` — монотонный таймер, точнее `time.time()` для latency
- `logger.exception(...)` в `except` — пишет stacktrace в лог и пробрасывает дальше, чтобы exception handler сделал HTTP-код

Куда это вырастет:

- В Б3.6 этот middleware переедет на `structlog` и будет писать JSON-логи
- `llm_cost` обновляется в `LLMService.complete` через `request.state` или contextvars

Middleware: порядок имеет значение

Middleware выполняется как LIFO-стек: добавили последним — выполнится первым.

```
example.python

1 # app/main.py
2 app = FastAPI(lifespan=lifespan)
3
4 # Добавляем В ПОРЯДКЕ внешний → внутренний
5 app.add_middleware(CORSMiddleware, ...) # 1. внешний
6 app.add_middleware(GZipMiddleware, minimum_size=1000) # 2. сжатие
7 app.middleware("http")(observability_middleware) # 3. request_id + logging
8 # 4. ← здесь начинается ваш endpoint
```


Middleware: порядок имеет значение

Поток запроса:

```
request  → CORS → GZip → Observability → [Router → Service → LLM]  
response ← CORS ← GZip ← Observability ← [Router ← Service ← LLM]
```

Middleware: порядок имеет значение

Правила порядка:

- **CORS** — всегда самый внешний (должен отвечать на OPTIONS до остальных)
- **Authentication** — перед business middleware
- **Observability** (request_id, tracing) — ближе к endpoint, чтобы ID был доступен везде
- **GZip** — после логирования (логи пишутся на несжатом ответе)

Middleware: CORS

Важное правило: `allow_origins=["*"]` несовместим с `allow_credentials=True`. Либо список конкретных доменов, либо без credentials.

```
example.python

1 # app/main.py (продолжение)
2 from fastapi.middleware.cors import CORSMiddleware
3
4 app.add_middleware(
5     CORSMiddleware,
6     allow_origins=get_settings().cors_origins,
7     allow_credentials=True,
8     allow_methods=["GET", "POST", "OPTIONS"],
9     allow_headers=["Content-Type", "Authorization", "X-Request-ID"],
10    expose_headers=["X-Request-ID", "X-LLM-Cost-USD"],
11 )
```

Exception handlers: домен → HTTP

example.python

```
1 # app/core/exceptions.py
2 class LLMError(Exception): ...
3 class LLMRateLimitError(LLMError): ...
4 class LLMAuthError(LLMError): ...
5 class LLMLTimeoutError(LLMError): ...
6 class LLMContentFilterError(LLMError): ...
```

example.python

```
1 # app/main.py (продолжение)
2 from fastapi import Request
3 from fastapi.responses import JSONResponse
4 from app.core.exceptions import (
5     LLMError, LLMRateLimitError, LLMAuthError,
6     LLMLTimeoutError, LLMContentFilterError,
7 )
8
9 _STATUS_MAP = {
10     LLMRateLimitError: (429, "llm_rate_limit"),
11     LLMAuthError: (502, "llm_auth_error"),
12     LLMLTimeoutError: (504, "llm_timeout"),
13     LLMContentFilterError: (400, "content_filter"),
14     LLMError: (502, "llm_error"),
15 }
```

Exception handlers: домен → HTTP

example.python

```
1 @app.exception_handler(LLMError)
2 async def handle_llm_error(request: Request, exc: LLMError):
3     for cls, (status, code) in _STATUS_MAP.items():
4         if isinstance(exc, cls):
5             return JSONResponse(
6                 status_code=status,
7                 content={"error": {"code": code, "message": str(exc)}},
8                 headers={"X-Request-ID": getattr(request.state, "request_id", "")},
9             )
```

Кеширование через Redis

Два подхода:

1. **Вручную внутри сервиса** — показан выше. Полный контроль ключа и TTL.
2. Через **aio-cache** декораторы:

Кеширование через Redis

```
example.python

1  from aiocache import cached, Cache
2  from aiocache.serializers import JsonSerializer
3
4  @cached(
5      ttl=3600,
6      cache=Cache.REDIS,
7      serializer=JsonSerializer(),
8      key_builder=lambda *args, **kwargs: hash_of(args[1]), # req — второй аргумент метода
9      endpoint="localhost", port=6379,
10 )
11 async def complete(self, req: ChatRequest) -> ChatResponse: ...
```

Кеширование через Redis

Что кешируем / что нет:

- Кешируем: **temperature=0**, детерминированные промпты, повторяющиеся классификации
- Не кешируем: streaming (бессмысленно), **temperature > 0.3** (вариативность важна), пользовательские данные в промпте

RequestValidationError: своё 422

По умолчанию FastAPI возвращает 422 с техническим форматом:

example.json

```
1 {"detail": [{"type": "missing", "loc": ["body", "messages"], "msg": "..."}]}
```

RequestValidationError: своё 422

Свой обработчик — красивее и унифицировано:

```
example.python

1 from fastapi.exceptions import RequestValidationError
2
3
4 @app.exception_handler(RequestValidationError)
5 async def handle_validation(request: Request, exc: RequestValidationError):
6     errors = [
7         {"field": ".".join(str(p) for p in e["loc"][1:]), "message": e["msg"]}
8         for e in exc.errors()
9     ]
10    return JSONResponse(
11        status_code=422,
12        content={"error": {"code": "validation_error", "fields": errors}},
13        headers={"X-Request-ID": getattr(request.state, "request_id", "")},
14    )
```

RequestValidationError: своё 422

Единый формат ошибок для всего API = счастливый фронтендер.

Swagger: примеры и описания

```
example.python

1  # app/schemas/chat.py
2  from pydantic import BaseModel, ConfigDict
3
4
5  class ChatRequest(BaseModel):
6      model_config = ConfigDict(
7          json_schema_extra={
8              "examples": [
9                  {
10                      "messages": [
11                          {"role": "system", "content": "Ты полезный ассистент."},
12                          {"role": "user", "content": "Напиши hello world на Python"},
13                      ],
14                      "model": "gpt-5-mini",
15                      "temperature": 0.2,
16                  }
17              ]
18          }
19      )
20      # ... поля
```

Swagger: примеры и описания

```
example.python

1  @router.post(
2      "",
3      response_model=ChatResponse,
4      summary="Синхронный чат",
5      description="Возвращает полный ответ LLM за один HTTP-запрос.",
6      responses={
7          200: {"description": "Успешный ответ"},
8          422: {"description": "Невалидный запрос"},
9          429: {"description": "Rate limit провайдера"},
10     },
11 )
12 async def chat_completions(...): ...
```

Swagger: примеры и описания

Swagger UI доступен на [/docs](#), ReDoc — на [/redoc](#).

Тесты и запуск в production

Сервис написан. Дальше — как мы его проверяем и как запускаем в prod.

Что было: сервис, endpoints, middleware, exception handlers, кеш, Swagger.

Что добавляем: async-тесты через `pytest-asyncio` + `httpx.AsyncClient` + `dependency_overrides`, запуск под `uvicorn/gunicorn`.

Тесты: почему pytest-asyncio и httpx

Почему не TestClient (sync):

- TestClient использует `requests` → не тестирует реальный async flow
- Blocking client не даёт протестировать SSE как stream
- Внутри всё равно оборачивает async в sync через threading

Почему AsyncClient + ASGITransport:

- Один event loop для теста и приложения — нет thread-surprise
- Можно писать `async for chunk in response.aiter_text():` для SSE
- Та же библиотека (httpx), что и в продакшене для внешних HTTP-вызовов

Почему pytest-asyncio в режиме auto:

- Не нужно помечать каждый async-тест декоратором
- `asyncio_mode = "auto"` — общепринятый стандарт

Альтернатива: anyio-backend. Работает и на trio, и на asyncio. Для FastAPI это избыточно — asyncio достаточно.

Тесты: conftest.py — фикстуры

Три фикстуры: `mock_llm`, `mock_cache`, `client` с подменой зависимостей.

example.python

```
1 # tests/conftest.py
2 import pytest
3 from unittest.mock import AsyncMock, MagicMock
4 from httpx import AsyncClient, ASGITransport
5
6 from app.main import app
7 from app.deps.providers import get_llm, get_cache
8
9
10 @pytest.fixture
11 def mock_llm():
12     llm = AsyncMock()
13     llm.chat.completions.create.return_value = MagicMock(
14         choices=[MagicMock(message=MagicMock(content="МОК-ОТВЕТ"))],
15         model="gpt-5-mini",
16         usage=MagicMock(prompt_tokens=10, completion_tokens=5, total_tokens=15),
17     )
18     return llm
```

example.python

```
1 @pytest.fixture
2 def mock_cache():
3     cache = AsyncMock()
4     cache.get.return_value = None
5     cache.setex.return_value = True
6     return cache
7
8
9 @pytest.fixture
10 async def client(mock_llm, mock_cache):
11     app.dependency_overrides[get_llm] = lambda: mock_llm
12     app.dependency_overrides[get_cache] = lambda: mock_cache
13     async with AsyncClient(
14         transport=ASGITransport(app=app),
15         base_url="http://test",
16     ) as ac:
17         yield ac
18     app.dependency_overrides.clear()
```

Тесты: conftest.py — что важно

Ключевые места:

- `AsyncMock` для async-методов OpenAI-клиента. Обычный `MagicMock` вернёт корутину, которую нужно `await`'ить — `pytest` ругнётся
- `ASGITransport` — `httpx` напрямую работает с ASGI-приложением, без реального TCP. Тесты быстрые и детерминированные
- `dependency_overrides[get_llm] = lambda: mock_llm` — главная польза DI: endpoint получает мок вместо реального клиента
- `app.dependency_overrides.clear()` в конце фикстуры — иначе тесты влияют друг на друга

Обязательная настройка в `pyproject.toml`:

Режим `auto` — каждая `async def test_*` становится async-тестом без декоратора `@pytest.mark.asyncio`.

```
example.toml
1 [tool.pytest.ini_options]
2 asyncio_mode = "auto"
```

Тесты: пять примеров на разные аспекты

```
example.python

1  # tests/test_chat.py
2  import pytest
3
4
5  @pytest.mark.asyncio
6  async def test_chat_ok(client, mock_llm):
7      resp = await client.post("/chat", json={
8          "messages": [{"role": "user", "content": "hi"}],
9      })
10     assert resp.status_code == 200
11     data = resp.json()
12     assert "content" in data
13     mock_llm.chat.completions.create.assert_awaited_once()
14
15
16  @pytest.mark.asyncio
17  async def test_chat_validation_422(client):
18     resp = await client.post("/chat", json={"messages": []})
19     assert resp.status_code == 422
```

Тесты: пять примеров на разные аспекты

```
example.python

1  @pytest.mark.asyncio
2  async def test_chat_max_tokens_limit(client):
3      resp = await client.post("/chat", json={
4          "messages": [{"role": "user", "content": "x"}],
5          "max_tokens": 99_999,
6      })
7      assert resp.status_code == 422
8
9
10 @pytest.mark.asyncio
11 async def test_health(client):
12     resp = await client.get("/health")
13     assert resp.status_code == 200
14     assert resp.json() == {"status": "ok"}
15
16
17 @pytest.mark.asyncio
18 async def test_batch_limit(client):
19     payload = [{"messages": [{"role": "user", "content": str(i)}]} for i in range(21)]
20     resp = await client.post("/chat/batch", json=payload)
21     assert resp.status_code == 413
```

Тесты: пять примеров на разные аспекты

Запуск: `pytest -v` или `uv run pytest -v`.

Тесты: что покрываем и куда расти

Что покрывают пять тестов:

- `test_chat_ok` — happy path. Mock возвращает ответ, endpoint отвечает 200
- `test_chat_validation_422` — пустой `messages` ловит Pydantic
- `test_chat_max_tokens_limit` — граница `Field(le=16_000)` срабатывает
- `test_health` — независимый endpoint, без моков
- `test_batch_limit` — ручная `HTTPException(413)` от нашего batch-limit

Куда расти для диплома:

- Стриминг — тест с mocked async generator (на следующем слайде)
- Exception handler — мок бросает `LLMRateLimitError`, ожидаем 429
- Middleware — `X-Request-ID` возвращается в заголовках

Частая ошибка: `assert_called_once()` вместо `assert_awaited_once()`. У `AsyncMock` метод `assert_awaited_once` проверяет именно `await`, не просто вызов корутины.

Test streaming-endpoint

example.python

```
1 # tests/test_stream.py
2 import pytest
3
4
5 @pytest.mark.asyncio
6 async def test_chat_stream(client, mock_llm):
7     # Настраиваем mock как async-генератор
8     async def fake_stream():
9         for text in ["При", "вет", "!"]:
10             yield MagicMock(
11                 choices=[MagicMock(delta=MagicMock(content=text))],
12                 usage=None,
13             )
14         yield MagicMock(choices=[], usage=MagicMock(
15             prompt_tokens=5, completion_tokens=3, total_tokens=8,
16         ))
17
18     mock_llm.chat.completions.create.return_value = fake_stream()
```

example.python

```
1     async with client.stream("POST", "/chat/stream", json={
2         "messages": [{"role": "user", "content": "hi"}]},
3     }) as resp:
4         assert resp.status_code == 200
5         assert resp.headers["content-type"].startswith("text/event-stream")
6
7         chunks = []
8         async for line in resp.aiter_lines():
9             if line.startswith("data: "):
10                 chunks.append(line[6:])
11
12     assert any("При" in c for c in chunks)
13     assert any("done" in c or "usage" in c for c in chunks)
```


Запуск в production: uvicorn vs gunicorn

Dev:

example.bash

```
1 uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```


Запуск в production: uvicorn vs gunicorn

Prod (вариант 1 — голый uvicorn):

```
example.bash
1  uvicorn app.main:app \
2    --host 0.0.0.0 --port 8000 \
3    --workers 4 \
4    --loop uvloop --http httptools \
5    --no-access-log \
6    --timeout-keep-alive 75
```

Запуск в production: uvicorn vs gunicorn

Prod (вариант 2 — gunicorn + UvicornWorker):

```
example.bash
1 gunicorn app.main:app \
2   --workers 4 \
3   --worker-class uvicorn.workers.UvicornWorker \
4   --bind 0.0.0.0:8000 \
5   --timeout 120 \
6   --graceful-timeout 30
```

Сквозные связи: куда это пойдёт дальше

Этот сервис — фундамент, который будет расти 3–4 месяца. Закладываем качество сразу.



Что мы НЕ делаем сегодня

- **Docker и docker-compose** — блок **3.5**. Сегодня запускаем через `uvicorn --reload`
- **Structured logging, Arize Phoenix, OpenTelemetry** — блок **3.6**. Сегодня стандартный `logging`
- **Глубокое тестирование, VCR.py, golden dataset, LLM-as-judge, CI** — блок **3.7**. Сегодня минимум unit-тестов
- **Prompt injection, garak, input/output validation, PII masking** — блок **3.8**. Сегодня нет защитной логики
- **Rate limiting per user, API-keys, OAuth2** — часть БЗ.8 + диплома
- **Prometheus метрики, Grafana** — опционально в БЗ.6
- **Полноценный RAG в /chat** — модуль **5**
- **Multi-agent в /chat** — модуль **6**

Сегодня мы строим скелет. В следующих блоках его обвешиваем всем, что нужно для продакшена.

Диаграмма: структура проекта

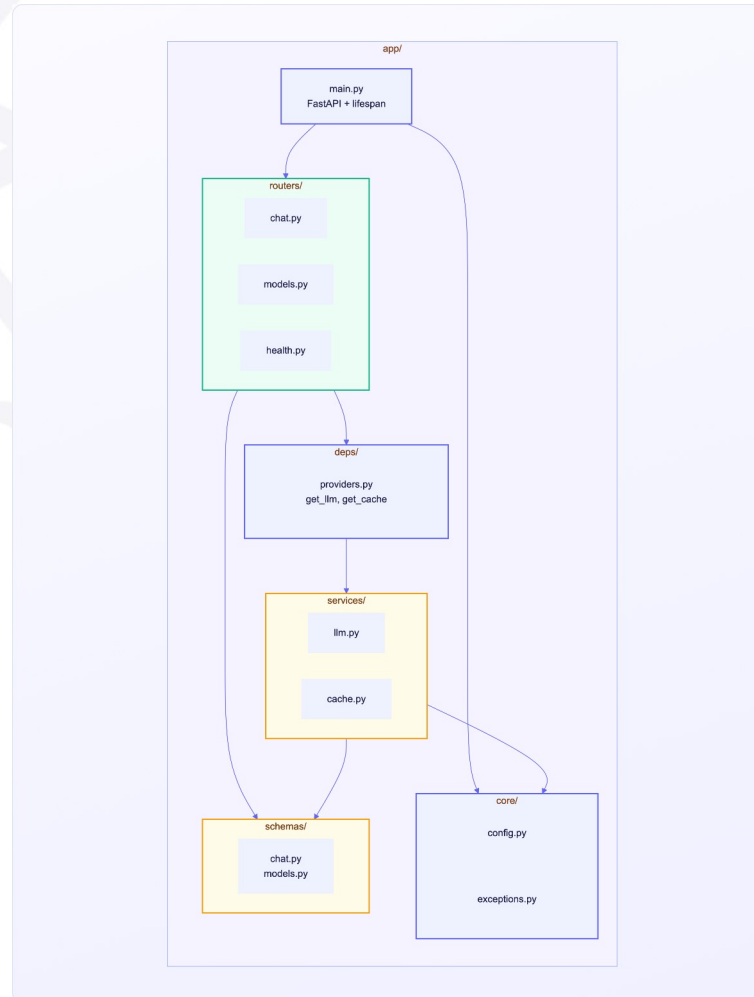


Диаграмма: жизненный цикл запроса

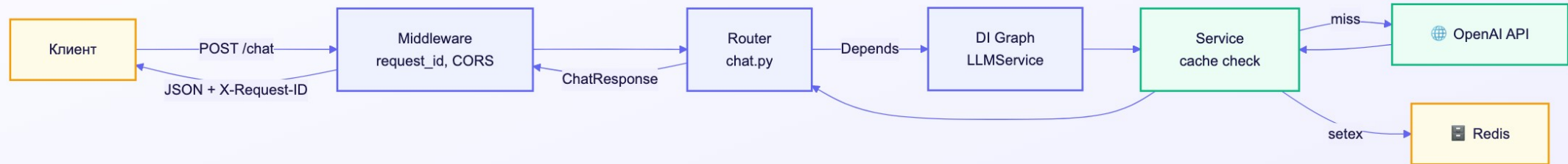
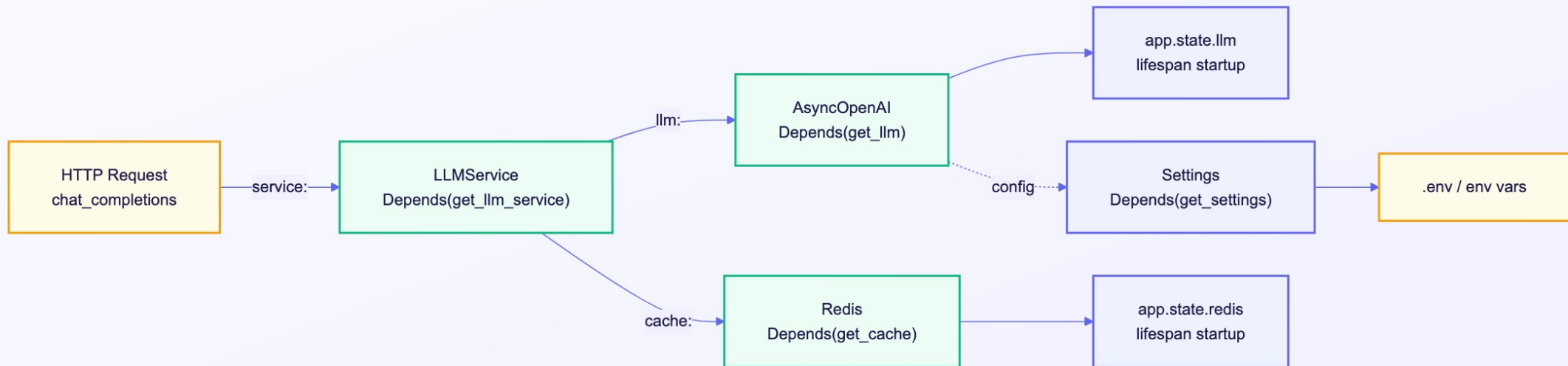


Диаграмма: DI-граф



Управление зависимостями: uv / poetry / pdm

Современный стандарт — uv (быстрее poetry в 10–100 раз).

example.toml

```
1  # pyproject.toml (uv)
2  [project]
3  name = "llm-service"
4  version = "0.1.0"
5  requires-python = ">=3.12"
6  dependencies = [
7      "fastapi[standard]>=0.110",
8      "pydantic>=2.6",
9      "pydantic-settings>=2.2",
10     "uvicorn[standard]>=0.29",
11     "openai>=2.0",
12     "redis>=5.0",
13     "tenacity>=8.2",
14     "structlog>=24.1",
15 ]
```

example.toml

```
1  [tool.uv]
2  dev-dependencies = [
3      "pytest>=8.0",
4      "pytest-asyncio>=0.23",
5      "httpx>=0.27",
6      "ruff>=0.4",
7      "mypy>=1.10",
8  ]
9
10 [tool.pytest.ini_options]
11 asyncio_mode = "auto"
```


Управление зависимостями: uv / poetry / pdm

Команды:

```
example.bash  
  
1 uv sync          # установить всё  
2 uv run pytest -v  # запустить тесты  
3 uv run uvicorn app.main:app --reload
```

Свяжем с прошлыми модулями ещё раз

Ничего нового в фундаменте. Новое — как это склеивается в HTTP-сервис через FastAPI.

Концепт сегодня	Откуда пришло
<code>AsyncOpenAI</code> клиент	M2B1 — базовый API-клиент
<code>tenacity</code> retry в сервисе	M2B3 — надёжность LLM-клиента
Redis-кеш по хешу промпта	M2B4 — кеширование ответов
Pydantic v2 схемы, <code>Field</code> , валидация	M1B3 — Structured Outputs
Слои Gateway / Service	M3B2 — архитектурные паттерны
<code>Semaphore</code> , <code>gather</code> в <code>/batch</code>	M3B3 — async + параллелизм
SSE через async-генератор	M3B3 — streaming
Расчёт стоимости по usage	M1B4 — экономика LLM

Частые грабли, которых не допустим

По конфигу:

- `allow_origins=["*"] + allow_credentials=True` — CORS не работает
- `.env` закоммичен в git — ключи утекли
- `Settings()` создаётся на каждый запрос без `@lru_cache`

По async:

- `time.sleep(5)` вместо `await asyncio.sleep(5)` — блок event loop
- В streaming не выставлен `stream_options={"include_usage": True}` — нет usage
- В batch нет `Semaphore` — 500 параллельных запросов, rate limit

По тестам:

- Забыли `asyncio_mode = "auto"` — тесты не запускаются
- Тест ходит в реальный OpenAI — деньги и время

Домашнее задание

Цель: собрать работающий FastAPI-сервис, который станет основой дипломного проекта.

Минимум (для зачёта):

1. Структура `app/{core, deps, routers, services, schemas}` — не один файл
2. Минимум **3 endpoints** из: `POST /chat`, `POST /chat/stream`, `POST /chat/batch`, `GET /models`, `GET /health`
3. Конфиг через `pydantic-settings` с `SecretStr` для `OPENAI_API_KEY`
4. DI через `Annotated[... , Depends(...)]`, не глобалы
5. `lifespan` для старта и остановки клиента
6. **Middleware**: `request_id` + `latency` + `CORS`
7. **Кеш ответов** (dict для MVP, Redis — предпочтительно)
8. **Swagger** с примерами (`json_schema_extra`)

Бонус:

- `/chat/stream` через `StreamingResponse`, работающий `curl -N`
- Exception handler для доменных LLM-ошибок с правильными HTTP-кодами
- Discriminated union в `ChatRequest` — задел под второй провайдер (Ollama, M1B3)
- Расчёт стоимости в response-заголовке `X-LLM-Cost-USD`

Дополнительные материалы для самостоятельного изучения

HTTP-клиенты и middleware:

- [Zapros](#) — современный Python HTTP-клиент, transport-agnostic (HTTP/1/2/3, Rust request, браузер через Pyodide).
Middleware-архитектура: cache, retry, mock, VCR-cassettes — без отдельных обёрток.

Dependency Injection:

- [python-injection](#) — DI на type-hints, 4 lifecycle'a, scoped через context manager. Если стандартный **Depends** начинает тесниться (много слоёв, тесты с подменой целых веток графа) — это аккуратная альтернатива.

Книги:

- [«FastAPI Cookbook» \(Giunio De Luca, Packt, 2024\)](#) — практические рецепты по аутентификации, фоновым задачам, тестированию, деплою. Хороший справочник на стол.

Референсы — FastAPI, Pydantic, streaming

FastAPI и экосистема:

- [FastAPI docs](#) — официальная документация
- [FastAPI Best Practices \(zhanyamkanov\)](#) — структура, паттерны, грабли
- [LangServe](#) — образец FastAPI-сервиса для LLM
- [sse-starlette](#) — SSE с keep-alive и disconnect detection

Pydantic и конфиг:

- [Pydantic v2 Concepts](#) — BaseModel, validators, unions
- [pydantic-settings](#) — конфиги из env

Streaming и async:

- [OpenAI Async](#)
- [Starlette — StreamingResponse](#) — формат SSE `data: ...\n\n`
- [Redis asyncio](#)

Референсы — тестирование и деплой

Тестирование:

- [pytest-asyncio](#)
- [httpx AsyncClient](#)

Управление зависимостями и деплой:

- [uv — быстрый pip/poetry-replacement](#)
- [Uvicorn Deployment](#)
- [Gunicorn + UvicornWorker](#)